

From Optimal Control to Diffusion Policy

A Trajectory-Optimization $\rightarrow$ TVLQR $\rightarrow$ Behavioural-Cloning Pipeline for
Double-Pendulum Swing-Up

Group Project TN2 — Technical Documentation

June 25, 2026

Contents

1	Introduction	3
2	Overview of the Pipeline	4
3	The Plant Model	5
4	Stage 1 — Trajectory Optimization and TVLQR (<code>kux_generation.py</code>)	6
4.1	The optimization problem	6
4.2	Time-varying LQR gains	6
4.3	Outputs	7
5	Stage 2 — Expert Dataset Generation (<code>generate_tv1qr_dataset_kux.py</code>)	8
5.1	Reference discovery and the hold gain	8
5.2	Two integration rates	8
5.3	Dagger-style noise, mirroring, and hold rollouts	8
5.4	Output format	8
6	Stage 3 — The Diffusion Policy Model (<code>diffusion_policy.py</code>)	10
6.1	Forward process and the scheduler	10
6.2	Noise-prediction backbones	10
6.3	The DiffusionPolicy class	10
7	Stage 4 — Training the Policy (<code>train_diffusion_policy.py</code>)	11
7.1	Feature engineering and conditioning	11
7.2	Architecture selection and checkpointing	11
8	Deviations from Vanilla Diffusion: The Implemented Tricks	12
8.1	Reformulation: what is actually diffused	12
8.2	Representation tricks that make the Gaussian prior appropriate	12
8.3	Training-loop tricks	12
8.4	Inference / sampling tricks	13
8.5	Backbone tricks	13
9	Interfaces and Data Contracts	14
10	Known Limitations	15
11	Conclusion	16

1 Introduction

Swinging an underactuated double pendulum from its stable hanging configuration up to an unstable equilibrium — and then *holding* it there — is a classic benchmark in nonlinear control. The task is hard for two reasons: the plant is strongly nonlinear and torque-limited, so the swing-up must pump energy through a carefully timed sequence of motions; and the upright goal is an unstable equilibrium, so balancing demands fast, accurate feedback. Reinforcement learning can solve such tasks but is sample-hungry and delicate to tune. This project takes a different route: *imitation learning*, in which a classical optimal-control expert is constructed first, demonstrations are harvested from it, and a generative neural policy is fit to those demonstrations.

This report documents four Python modules that together implement that pipeline end to end for a system-identified double pendulum. The pipeline proceeds in four stages: (i) `kux_generation.py` solves a family of direct-collocation trajectory-optimization problems and computes time-varying LQR (TVLQR) feedback gains about each optimal trajectory; (ii) `generate_tvlqr_data` rolls those TVLQR controllers out in closed loop through the identified plant to synthesise an expert demonstration dataset; (iii) `diffusion_policy.py` defines a conditional Denoising Diffusion Probabilistic Model (DDPM) over action sequences with both MLP and Transformer backbones; and (iv) `train_diffusion_policy.py` trains that diffusion policy by behavioural cloning on the expert dataset.

The remainder of the report is organised as follows. Section 2 gives a high-level overview of the pipeline and the file-on-disk interfaces that decouple its stages, and Section 3 specifies the shared plant model. Sections 4–7 then describe the four stages in turn: the optimal-control expert and its TVLQR stabilisation, the closed-loop dataset generation, the diffusion-policy model, and its training by behavioural cloning. Section 8 isolates the modifications that adapt a textbook DDPM into a closed-loop controller for an unstable, torque-limited plant. Finally, Section 9 collects the data contracts between stages, Section 10 records the known limitations of the resulting controller, and Section 11 concludes.

2 Overview of the Pipeline

The goal is to learn a neural feedback policy that swings a double pendulum from the hanging-down configuration up to an unstable equilibrium (upright, or the folded $[\pi, \pi]$ pose) and holds it there. Rather than learning from scratch with reinforcement learning, the project uses *imitation learning*: a classical optimal-control expert is constructed, demonstrations are harvested from it, and a generative policy is fit to those demonstrations.

Table 1: The four modules and their roles in the pipeline. Paths are relative to the project root; the first two modules live in `double_pendulum/`.

Module	Stage	Responsibility
<code>generators/ kux_generation.py</code>	Expert design	Solves the direct-collocation trajectory-optimization problem (IPOPT) and the TVLQR backward pass. Writes one folder of (x, u, K) per (Q, R, x_{goal}) configuration.
<code>generators/ generate_tvlqr_ dataset_kux.py</code>	Data harvesting	Rolls every TVLQR reference (and its mirror) out in closed loop through the plant, logging (state, action) pairs to an HDF5 dataset.
<code>diffusion_models/ diffusion_policy.py</code>	Model	Defines the DDPM scheduler, the MLP and Transformer noise-prediction networks, and the <code>DiffusionPolicy</code> train/sample/act class.
<code>train_diffusion_ policy.py</code>	Learning	Slices trajectories into conditioning/action windows, normalizes them, and trains the diffusion policy; writes a self-describing checkpoint.

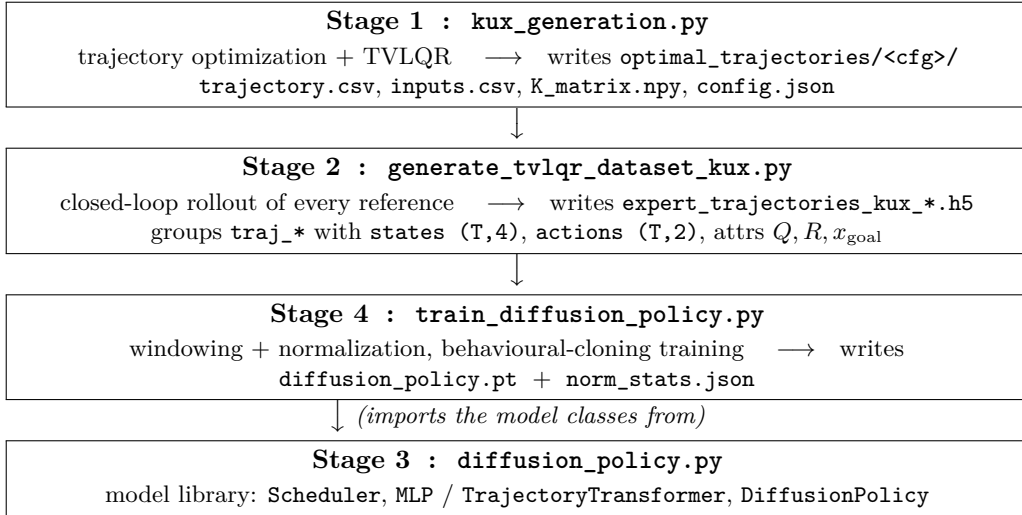


Figure 1: End-to-end data flow. Each downward arrow between Stages 1, 2 and 4 is a file-on-disk interface, so the stages are decoupled and independently runnable. Stage 3 is the shared model library that the trainer (and the evaluation script) build their networks from.

The state is $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^\top \in \mathbb{R}^4$ and the control is the joint-torque vector $u = [u_1, u_2]^\top \in \mathbb{R}^2$, bounded by a hard actuator limit $|u_i| \leq \tau_{\max} = 0.07$ Nm.

3 The Plant Model

All four stages share one plant: the system-identified “cloudpendulum” model. `kux_generation.py` encodes it directly; the dataset generator imports the same parameters `P` from `rollout_tvqlqr.py`. The continuous dynamics follow the standard manipulator equation

$$M(x)\ddot{q} + C(x)\dot{q} + f(\dot{q}) = u + G(x), \quad (1)$$

with $q = [q_1, q_2]^\top$. The mass matrix, Coriolis matrix and gravity vector are

$$M(x) = \begin{bmatrix} I_1 + I_2 + l_1^2 m_2 + 2l_1 m_2 l_{c2} \cos q_2 + g_{r1}^2 I_r + I_r & I_2 + l_1 m_2 l_{c2} \cos q_2 \\ I_2 + l_1 m_2 l_{c2} \cos q_2 & I_2 \end{bmatrix}, \quad (2)$$

$$C(x) = \begin{bmatrix} -2\dot{q}_2 k \sin q_2 & -\dot{q}_2 k \sin q_2 \\ \dot{q}_1 k \sin q_2 & 0 \end{bmatrix}, \quad k = l_1 m_2 l_{c2}, \quad (3)$$

$$G(x) = \begin{bmatrix} -g m_1 l_{c1} \sin q_1 - g m_2 (l_1 \sin q_1 + l_{c2} \sin(q_1 + q_2)) \\ -g m_2 l_{c2} \sin(q_1 + q_2) \end{bmatrix}. \quad (4)$$

Friction is a viscous term plus an arctan-smoothed Coulomb term (the smooth surrogate for $\text{sign}(\dot{q})$, with `FRICTION_SCALE`= 100):

$$f(\dot{q}) = \begin{bmatrix} b_1 \dot{q}_1 + \mu_1 \arctan(100 \dot{q}_1) \\ b_2 \dot{q}_2 + \mu_2 \arctan(100 \dot{q}_2) \end{bmatrix}. \quad (5)$$

Solving (1) for the accelerations gives the first-order field $\dot{x} = f(x, u) = [\dot{q}, M(x)^{-1}(u + G(x) - C(x)\dot{q} - f(\dot{q}))]$, implemented in `dynamics(x,u)` and integrated with a fixed-step RK4 `rk4_step`. The identified parameters are listed in Table 2.

Table 2: System-identified model parameters `P` (`kux_generation.py`).

$m_1 = 0.1055$ kg	$m_2 = 0.0762$ kg	$l_1 = l_2 = 0.05$ m	$l_{c1} = 0.05$ m
$l_{c2} = 0.0367$ m	$I_1 = 4.62 \times 10^{-4}$	$I_2 = 2.37 \times 10^{-4}$	$g = 9.81$
$b_1 \approx 0$	$b_2 = 5.1 \times 10^{-4}$	$\mu_1 = 3.05 \times 10^{-3}$	$\mu_2 = 7.8 \times 10^{-4}$
$g_{r1} = 403$	$g_{r2} = 379$	$I_r = 0$	$\tau_{\max} = 0.07$ Nm

Symmetry. The plant is invariant under the left–right reflection $x \mapsto -x$, $u \mapsto -u$: because the kinematics enter only through $\sin / \cos$ and friction is odd in velocity, $f(-x, -u) = -f(x, u)$. Hence any valid trajectory $(x(t), u(t))$ has a mirror twin $(-x(t), -u(t))$ that swings up to the *same* goal from the other side, and the TVLQR gains K are identical at the mirrored point. The dataset generator exploits this to double its demonstrations for free (Section 5).

4 Stage 1 — Trajectory Optimization and TVLQR (`kux_generation.py`)

For each cost configuration this module (1) solves a direct-collocation trajectory-optimization problem with IPOPT to obtain the optimal state/control trajectories (x, u) , then (2) linearises about that trajectory and runs a backward Riccati pass to obtain the time-varying feedback gains K .

4.1 The optimization problem

With horizon $T = 4$ s and step $dt = 0.05$ s, the trajectory is discretised into $N = T/dt + 1 = 81$ nodes. The decision vector stacks all states then all controls,

$$z = [x_0, \dots, x_{N-1}, u_0, \dots, u_{N-1}] \in \mathbb{R}^{Nn_x + Nn_u}, \quad n_x = 4, \quad n_u = 2.$$

The cost is a quadratic running cost with a separate terminal weight,

$$J(z) = \frac{1}{2} \sum_{k=0}^{N-1} \left[(x_k - x_g)^\top Q (x_k - x_g) + u_k^\top R u_k \right] + (x_{N-1} - x_g)^\top Q_{\text{fin}} (x_{N-1} - x_g), \quad (6)$$

implemented in `total_objective`; its closed-form gradient `total_objective_deriv` is supplied to IPOPT analytically (the per-node gradients $2Q(x_k - x_g)$, $2Ru_k$ are computed with `jax.vmap`). The dynamics are imposed as equality constraints by *trapezoidal collocation* between neighbouring nodes,

$$x_{k+1} - x_k - \frac{dt}{2} (f(x_k, u_k) + f(x_{k+1}, u_{k+1})) = 0, \quad (7)$$

together with the boundary constraints $x_0 = x_0^{\text{start}}$ and $x_{N-1} = x_g$. The full constraint vector and its Jacobian (via `jax.jacobian`) are assembled in `constraints`. Controls are box-bounded to $\pm\tau_{\text{max}}$; states are free.

```
1 def trapezoidal_collocation(x_k, x_kp1, u_k, u_kp1, dynamics, dt):
2     f_k = dynamics(x_k, u_k)
3     f_kp1 = dynamics(x_kp1, u_kp1)
4     return (x_kp1 - x_k - (dt / 2.0) * (f_k + f_kp1)).flatten()
```

Listing 1: Trapezoidal-collocation defect (the dynamics constraint between two nodes).

The `Problem` class bakes a specific $(Q, R, Q_{\text{fin}}, x_0, x_g)$ into JIT-compiled JAX callbacks (`objective`, `gradient`, `constraints`, `jacobian`) that conform to the `cyipopt.Problem` interface. A linear interpolation $x_0 \rightarrow x_g$ with zero controls warm-starts the solver (`solve_trajectory`). IPOPT runs with `max_iter=100`, `tol=10-3`.

4.2 Time-varying LQR gains

Stabilising the optimal trajectory requires feedback. The module discretises the linearised dynamics *through the RK4 integrator* by differentiating each RK4 stage with the chain rule (`get_discrete_matrices`), producing the discrete sensitivity matrices A_d, B_d at every node — this is more faithful than an Euler discretisation of the continuous Jacobians A, B obtained from `jax.jacfwd`. The finite-horizon discrete Riccati recursion is then swept backward from the terminal cost $P_{N-1} = Q_{\text{fin}}$:

$$S_k = R + B_d^\top P_{k+1} B_d, \quad (8)$$

$$K_k = S_k^{-1} B_d^\top P_{k+1} A_d, \quad (9)$$

$$P_k = Q + A_d^\top P_{k+1} (A_d - B_d K_k). \quad (10)$$

The backward sweep is vectorised with `jax.lax.scan` over the reversed trajectory (`compute_tvlqr_gains`), yielding gains $K \in \mathbb{R}^{(N-1) \times n_u \times n_x}$.

4.3 Outputs

Each configuration is written to its own folder named by the Q/R diagonals and the goal (e.g. `Q_100_100_100_100__R_0p01_0p01__xgoal_3p142_0_0_0/`), containing `trajectory.csv` (x), `inputs.csv` (u), `K_matrix.npy` (K), and `config.json` (metadata, including the IPOPT status and the achieved final state). The configuration sweep fixes $Q = \text{diag}(100, 100, 100, 100)$ and sweeps the control weight $R \in \{0.01, 10, 50, 100, 200, 500\} \cdot I_2$ across two goals $x_g \in \{[\pi, 0, 0, 0], [\pi, \pi, 0, 0]\}$ — twelve references in total.

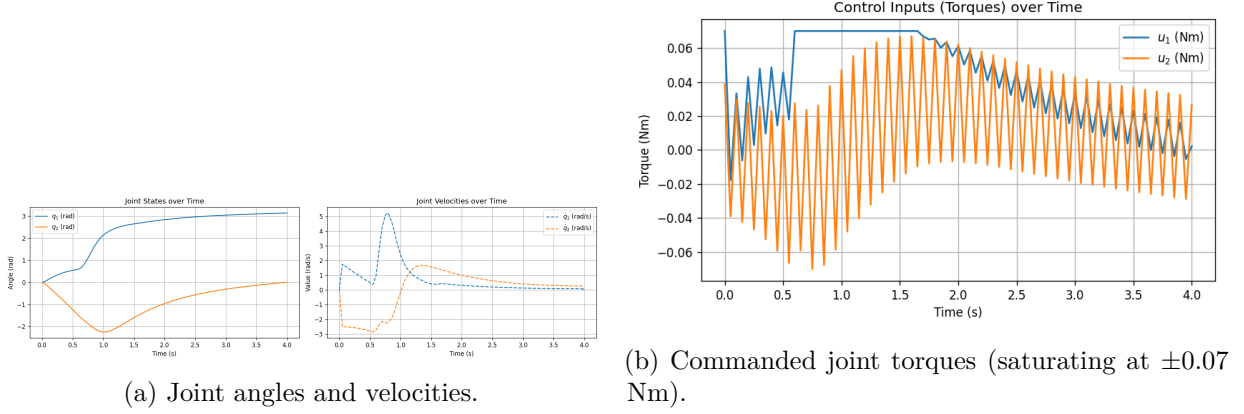


Figure 2: Example optimal swing-up to the upright goal $x_g = [\pi, 0, 0, 0]$ for the low control-weight configuration $R = 0.01 I_2$. Plots are emitted by `save_plots`.

5 Stage 2 — Expert Dataset Generation (`generate_tvqlqr_dataset_kux.py`)

This module turns the open-loop references of Stage 1 into a behavioural-cloning dataset by *rolling out* the TVLQR controllers in closed loop through the identified plant and logging the (observed state, commanded torque) stream at the control rate $dt_{\text{ctrl}} = 0.05$ s. Because every logged pair is produced by simulating the actual plant, each sample is dynamically valid by construction.

5.1 Reference discovery and the hold gain

`discover_configs` scans `optimal_trajectories/` for every folder holding the full quartet (`trajectory.csv`, `inputs.csv`, `K_matrix.npy`, `config.json`); `load_reference` reads back $(x_{\text{ref}}, u_{\text{ref}}, K, Q, R, x_g)$.

A subtlety: the saved K is the *finite-horizon* TVLQR gain, which ends with the swing-up and cannot hold the unstable inverted goal past the last node. The generator therefore solves the *infinite-horizon* LQR at the goal once, `K_hold = lqr_gain(xgoal, Q, R, dt=dt_sim)`, and hands it to every rollout so the upright hold persists. By the plant symmetry the same K_{hold} stabilises the mirrored goal $-x_g$. Critically the hold gain is designed for the *inner simulation step* dt_{sim} , not the control step — the inverted hold is not stabilisable at the 20 Hz control rate, so the rollout re-closes the hold loop at dt_{sim} .

5.2 Two integration rates

The references are 41-node, 0.05 s trajectories whose swing-ups are violent ($\dot{q}_2 > 40$ rad/s). Integrating the closed loop at the full 0.05 s control step destabilises tracking, so the rollout sub-steps RK4 at $dt_{\text{sim}} = 0.005$ s (the `--dt-sim` flag) while still commanding and *logging* actions at the 0.05 s control rate. This separation of the control rate from the integration rate is what makes the otherwise-fragile references trackable.

5.3 DAgger-style noise, mirroring, and hold rollouts

Three mechanisms broaden the state coverage of the demonstrations:

DAgger noise. On a fraction of swing-up rollouts (every $\sim 2/3$ of side-pairs) state noise of std 0.05 is injected for coverage of off-nominal states, but the *clean* commanded torque is logged as the supervised target. The noise latches *off* once the rollout first reaches the goal (`noise_until_upright=True`) so the fragile hold stays clean.

Mirroring. Each reference is also rolled out mirrored, $(-x_{\text{ref}}, -u_{\text{ref}})$, swinging up to the same goal from the other side. Episodes alternate sides (+1/−1) so the kept set is left–right balanced; each rollout records its `side` attribute.

Goal-hold rollouts. In `mode=both`, additional episodes start *perturbed* around the goal equilibrium and roll out the stabilising gain, recording the deviation $\rightarrow$ corrective-torque map. Without these the policy would only ever see the exact fixed point and could never learn to recover from small deviations.

Every reference (and its mirror) is self-checked — it must still swing up and hold in the plant model — before use; a reference that fails is skipped with a warning rather than aborting the whole sweep.

5.4 Output format

A single HDF5 file is written (e.g. `expert_trajectories_kux_swingup_hold_mirrored.h5`) with one group `traj_{idx}` per kept rollout. Group indices are global across all configurations. Each group holds `states` (T,4) and `actions` (T,2), plus per-group attributes that record the

expert identity: Q , R , x_{goal} , the source config folder name, the initial state, the injected noise std, the **side**, and the **kind** (**swingup/hold**). This is exactly the format the trainer consumes, and the x_{goal} attribute is what later enables (in principle) goal-conditioning.

6 Stage 3 — The Diffusion Policy Model (`diffusion_policy.py`)

The policy is a conditional DDPM over short action sequences, after Chi et al. (2023). It *denoises* a sampled Gaussian into an action chunk $a \in \mathbb{R}^{H \times n_u}$, conditioned on a clean (un-noised) context vector. The module provides a scheduler, two interchangeable backbones, and the training/sampling logic.

6.1 Forward process and the scheduler

Scheduler uses a linear β schedule from $\beta_0 = 3 \times 10^{-4}$ to $\beta_T = 0.03$ over $T = 100$ steps, with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$. The forward (noising) process samples

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (11)$$

returning both x_t and the noise ε (`add_noise`). Only the *actions* are noised; the conditioning context is never noised.

6.2 Noise-prediction backbones

Both backbones share the contract `forward(a_noisy, t, cond) -> predicted_noise`, so they are drop-in replacements.

MLP. Flattens the action chunk to $H \cdot n_u$, embeds the scalar timestep and the conditioning vector with small SiLU MLPs, concatenates, and maps through a 3-hidden-layer SiLU network back to $H \cdot n_u$. It ignores the temporal structure of the action chunk.

TrajectoryTransformer. Treats the H action steps as a sequence of tokens with learned positional embeddings, and *prepends* two context tokens — a sinusoidal timestep embedding and the conditioning embedding — so every action token attends to them. A pre-norm **TransformerEncoder** (GELU) processes the length- $(H+2)$ sequence; only the H action positions are projected back to n_u . This preserves the temporal structure the MLP discards.

6.3 The DiffusionPolicy class

This wraps a scheduler and a backbone and exposes training, sampling, and a control-convenience method.

train. For each batch it draws $t \sim \mathcal{U}\{1, \dots, T\}$, noises the actions, and minimises the noise-prediction MSE $\|\varepsilon - \varepsilon_\theta(x_t, t, \text{cond})\|^2$. It uses Adam with a per-step cosine-annealing LR schedule, automatic mixed precision on CUDA, and maintains an **exponential moving average** (EMA, decay 0.999) of the weights.

sample. Runs reverse DDPM ancestral sampling for one action chunk per conditioning vector. The flag `stochastic=False` drops the per-step ancestral noise (returns the posterior mean each step), which sharply lowers action variance — important for stabilising the unstable upright hold, where injected sampling noise would perturb the torque and topple the pendulum.

act. A closed-loop convenience wrapper: it takes the last k observed states (and optional goal), builds the conditioning vector, samples a chunk, and returns the first `n_exec` actions.

Why EMA for control. The EMA copy is a smoother estimate of the trained function than the last-step weights. Near the unstable upright equilibrium this yields a more *consistent* learned gain (less jitter), which is what lets the policy balance. The trainer saves the EMA weights as the inference checkpoint.

7 Stage 4 — Training the Policy (`train_diffusion_policy.py`)

This module loads the HDF5 expert dataset, converts it into supervised (`cond`, `action_seq`) windows, and trains the diffusion policy.

7.1 Feature engineering and conditioning

Raw angles are mapped to wrap-free angular features to avoid the 2π discontinuity, and velocities/actions are min-max normalised to $[-1, 1]$ (stats saved to `norm_stats.json` for inference):

$$x \mapsto [\sin q_1, \cos q_1, \sin q_2, \cos q_2, \hat{q}_1, \hat{q}_2] \in \mathbb{R}^6 \quad (\text{NX_FEAT} = 6). \quad (12)$$

For every time index i in a trajectory the dataset forms a conditioning vector from the last $k = 6$ feature vectors (left-padded at the start) and, when `USE_GOAL=True`, appends the per-trajectory goal feature read from the HDF5 `xgoal` attribute:

$$\text{cond} = [\text{hist}_{i-k+1:i}] \parallel \text{goal_feat} \in \mathbb{R}^{k \cdot 6 (+6)}.$$

The target is the next $H = 10$ normalised actions (right-padded by repetition at the trajectory's end).

7.2 Architecture selection and checkpointing

The architecture is CLI-selectable (`--arch mlp|transformer`, default `transformer`) and **self-describing in the checkpoint**: the network class name and its exact kwargs are stored under `config["net_kwargs"]`, so the evaluation script can rebuild the identical network with no manual edits. The checkpoint stores both the EMA weights (`model_state`, used for inference) and the raw weights (`model_state_raw`), along with all conditioning metadata (k , n_x , `nx_feat`, `use_goal`, horizon, timesteps). Periodic per-epoch snapshots are written every `CKPT_EVERY = 20` epochs.

Table 3: Default training hyperparameters (`train_diffusion_policy.py`).

History $k = 6$	Horizon $H = 10$	Diffusion steps $T = 100$	Epochs = 60
Batch = 512	LR = 10^{-4} (cosine)	Seed = 42	EMA decay = 0.999
MLP hidden = 512	TF $d_{\text{model}} = 192$	TF heads/layers = 4/5	TF dropout = 0.2

Note on synthetic holds. An older mechanism that synthesised hold samples by repeating each trajectory's final state/action is now disabled (`HOLD_STEPS = 0`): it taught the exact fixed point but not how to recover. Real hold demonstrations now come from the Stage-2 goal-hold rollouts, which capture the deviation→corrective-torque map directly.

8 Deviations from Vanilla Diffusion: The Implemented Tricks

A textbook DDPM (Ho et al. 2020) is *unconditional*: it samples $t \sim \mathcal{U}\{1, \dots, T\}$, noises the data x_0 , trains a network to predict the noise under MSE, and generates by ancestral reverse sampling from $\mathcal{N}(0, I)$. The pipeline keeps that backbone but layers a number of modifications that adapt it from image-style generation to *closed-loop control of an unstable, torque-limited plant*. Table 4 summarises them; the subsections below explain each and point to where it lives in the code.

Table 4: Vanilla DDPM vs. the control-oriented diffusion policy implemented here.

Aspect	Vanilla DDPM	This implementation
What is diffused	the whole data sample (e.g. an image), unconditional	an <i>action chunk</i> $a \in \mathbb{R}^{H \times n_u}$, conditioned on clean state history + goal
Diffusion length	$T \approx 1000$, $\beta \in [10^{-4}, 2 \times 10^{-2}]$	short $T = 100$, tuned $\beta \in [3 \times 10^{-4}, 3 \times 10^{-2}]$
Timestep input	integer index t	normalized scalar $t/T \in (0, 1)$
Inference weights	last-step weights	EMA weights (decay 0.999)
Reverse sampling	always stochastic (ancestral noise)	optional <i>deterministic</i> (posterior mean) for the unstable hold
Execution	generate once	receding horizon: predict H , apply first n_{exec}

8.1 Reformulation: what is actually diffused

Conditional, action-only diffusion. Only the actions are noised; the conditioning context (state history + goal) is passed *clean* and is never corrupted (`train: z_t, eps = scheduler.add_noise(act, t)`; `cond` bypasses the scheduler). This turns the generative model into a policy $p_\theta(a \mid \text{cond})$ rather than an unconditional sampler — the Diffusion-Policy formulation.

Action chunking (sequence diffusion). The diffused variable is a whole H -step action sequence, not a single action. The network treats the chunk as a unit, which lets it produce temporally consistent, multi-modal plans. At run time `act` executes only the first `n_exec` actions and re-plans — a receding-horizon scheme, not one-shot generation.

8.2 Representation tricks that make the Gaussian prior appropriate

DDPM denoises toward an $\mathcal{N}(0, I)$ -scaled target, so the data must be put on that scale (`train_diffusion_policy`).

Angular (wrap-free) features. Raw angles are replaced by $[\sin q_1, \cos q_1, \sin q_2, \cos q_2]$ so the network never sees a 2π discontinuity in the conditioning (`_state_to_features`).

Min-max normalization to $[-1, 1]$. Velocities and the diffused *actions* are linearly rescaled to $[-1, 1]$ (`_norm_action`); this matches the unit scale the diffusion prior assumes and keeps the noise schedule meaningful. The exact min/max are saved to `norm_stats.json` and inverted at inference (`denorm_action`).

History + goal conditioning. The context is the last k feature vectors concatenated with the per-trajectory goal feature, with left-padding at the start of a trajectory and right-padding (repeat the last action) at the end.

8.3 Training-loop tricks

EMA weights as the inference model. An exponential moving average of the parameters (decay 0.999) is maintained every step (`_update_ema`) and is what `save_checkpoint` stores

as `model_state`. The EMA copy is a smoother estimate of the trained function; near the unstable upright equilibrium this yields a more *consistent* learned gain (less jitter), which is what allows the policy to balance at all.

Short, tuned noise schedule. $T = 100$ steps (not ~ 1000) with a linear β from 3×10^{-4} to 3×10^{-2} keeps reverse sampling cheap enough to run inside a control loop while still denoising cleanly (`Scheduler`).

Normalized continuous timestep. The network is fed $t/T \in (0,1)$ rather than the integer index (`t_normalized = t / timesteps`), so the timestep embedding sees a bounded, scale-free input. (The Transformer’s sinusoidal embedding multiplies this by 1000 to recover useful frequencies.)

Restricted t range. Training draws $t \sim \mathcal{U}\{1, \dots, T-1\}$ (`torch.randint(1, timesteps)`): $t = 0$ is the clean sample (nothing to denoise) and is skipped.

Per-step cosine-annealing LR. The LR follows a cosine schedule advanced *every batch*, with $T_{\max} = \text{epochs} \times |\text{loader}|$ (`_lr_scheduler.step()` inside the batch loop), rather than the constant or per-epoch LR of a minimal trainer.

Mixed-precision training. On CUDA the forward/backward run under `autocast` with a `GradScaler` — a speed optimization that leaves the algorithm unchanged.

8.4 Inference / sampling tricks

Deterministic sampling — the headline control trick. The reverse process takes the DDPM posterior *mean* at each step,

$$a_{t-1} = \frac{1}{\sqrt{1-\beta_t}} a_t - \frac{\beta_t}{\sqrt{1-\beta_t} \sqrt{1-\bar{\alpha}_t}} \varepsilon_{\theta}(a_t, t, \text{cond}), \quad (13)$$

and with `stochastic=False` *omits* the ancestral noise term entirely. Vanilla DDPM always injects $\sqrt{\beta_t} z$; here that injected torque noise would perturb the command and topple the pendulum, so for the unstable upright hold the posterior mean is used to drive the action variance toward zero.

Fixed-large posterior variance. When stochasticity *is* wanted, the added noise uses $\sigma_t = \sqrt{\beta_t}$ (the “fixed-large” variance), and only for $t > 1$ — the final denoising step ($t = 1$) is always noise-free (`sample`).

Receding-horizon execution. `act` predicts the full horizon but returns only the first `n_exec` actions, so the policy re-conditions on fresh observations frequently instead of committing to a whole open-loop chunk.

8.5 Backbone tricks

A shared `forward(a_noisy, t, cond)` contract lets the MLP and the Transformer be swapped with a one-line change, and makes the choice self-describing in the checkpoint.

Transformer conditioning by context tokens. Instead of concatenating t and `cond` into the input, the Transformer *prepends* them as two extra tokens so every action token attends to them; only the H action positions are projected back to n_u . It uses pre-norm GELU encoder layers and small-init learned positional embeddings.

9 Interfaces and Data Contracts

The stages communicate only through files, which keeps them decoupled (Figure 1). The contracts are:

- **Stage 1 $\rightarrow$ 2:** a reference folder `{trajectory.csv ( $N, 4$ ), inputs.csv ( $N, 2$ ), K_matrix.npy ( $N-1, 2, 4$ ), config.json}`. The `load_reference` reader in Stage 2 mirrors the `save_trajectory` writer in Stage 1.
- **Stage 2 $\rightarrow$ 4:** an HDF5 file of `traj_*` groups, each `states (T,4) + actions (T,2)` with `Q/R/xgoal` attributes. Stage 4's `DiffusionPolicyDataset` reads exactly this layout.
- **Stage 4 $\rightarrow$ 3 (inference):** a self-describing checkpoint (`net_kwargs`, EMA weights) plus `norm_stats.json`. The `build_network/save_checkpoint` pair is the single source of truth that lets evaluation rebuild the model from Stage 3 classes.

10 Known Limitations

The documentation in the modules and the project’s working notes record several honest caveats about the current controller:

- **Goal-conditioning is presently nominal.** Although the dataset tags each demonstration with its x_{goal} and the trainer appends a goal feature, the learned policy can effectively ignore its goal input when the demonstrations are dominated by a single goal — the goal feature must actually *vary* across the dataset (multiple x_g in the sweep) for conditioning to take effect.
- **Closed-loop behavioural-cloning fragility.** A policy that imitates the TVLQR expert can still drift in closed loop due to covariate shift (the policy visits states the expert rarely demonstrated). The DAgger noise, mirroring, and goal-hold rollouts of Stage 2 are explicitly there to mitigate this, but they do not eliminate it.
- **The inverted hold needs a fast rate.** The unstable upright hold is not stabilisable at the 20 Hz control rate; Stage 2 re-closes the hold loop at the inner dt_{sim} , and a policy that only commands at 20 Hz cannot exactly reproduce that fast feedback. Deterministic sampling (`stochastic=False`) and the EMA weights are the model-side mitigations.

11 Conclusion

The four modules form a clean, file-decoupled imitation-learning pipeline: classical trajectory optimization and TVLQR build a provably-valid expert (`kux_generation.py`); closed-loop rollouts with symmetry, DAgger noise and goal-hold coverage harvest demonstrations (`generate_tvlqr_dataset_kux.py`); a conditional diffusion model with MLP/Transformer backbones provides an expressive multi-modal action generator (`diffusion_policy.py`); and a self-describing trainer fits it by noise-prediction behavioural cloning (`train_diffusion_policy.py`). The design choices throughout — analytic gradients, RK4-consistent TVLQR discretisation, the separation of control and integration rates, EMA weights, and deterministic sampling for the unstable hold — are all driven by the difficulty of stabilising an underactuated, torque-limited double pendulum at an unstable equilibrium.